

predict future sales

June 1, 2026

```
[1]: import numpy as np
import pandas as pd
```

```
[2]: train_data = pd.read_csv('competitive-data-science-predict-future-sales/
    ↪sales_train.csv')
sale_item = pd.read_csv('competitive-data-science-predict-future-sales/items.
    ↪csv')
item_categories = pd.read_csv('competitive-data-science-predict-future-sales/
    ↪item_categories.csv')
shop_name = pd.read_csv('competitive-data-science-predict-future-sales/shops.
    ↪csv')
test_data = pd.read_csv('competitive-data-science-predict-future-sales/test.
    ↪csv')
```

```
[3]: train_data
```

```
[3]:
```

	date	date_block_num	shop_id	item_id	item_price	\
0	02.01.2013	0	59	22154	999.00	
1	03.01.2013	0	25	2552	899.00	
2	05.01.2013	0	25	2552	899.00	
3	06.01.2013	0	25	2554	1709.05	
4	15.01.2013	0	25	2555	1099.00	
...	...	...	...	...	...	
2935844	10.10.2015	33	25	7409	299.00	
2935845	09.10.2015	33	25	7460	299.00	
2935846	14.10.2015	33	25	7459	349.00	
2935847	22.10.2015	33	25	7440	299.00	
2935848	03.10.2015	33	25	7460	299.00	

	item_cnt_day
0	1.0
1	1.0
2	-1.0
3	1.0
4	1.0
...	...
2935844	1.0

```

2935845      1.0
2935846      1.0
2935847      1.0
2935848      1.0

```

[2935849 rows x 6 columns]

```

[4]: def split_month(columns):
      word= columns.split('.')[1]
      return word

train_data['Month']=train_data['date'].apply(split_month)

```

```

[5]: train_data

```

```

[5]:
      date  date_block_num  shop_id  item_id  item_price  \
0    02.01.2013             0      59    22154      999.00
1    03.01.2013             0      25     2552      899.00
2    05.01.2013             0      25     2552      899.00
3    06.01.2013             0      25     2554     1709.05
4    15.01.2013             0      25     2555     1099.00
...      ...             ...      ...      ...      ...
2935844  10.10.2015          33      25     7409      299.00
2935845   09.10.2015          33      25     7460      299.00
2935846  14.10.2015          33      25     7459      349.00
2935847  22.10.2015          33      25     7440      299.00
2935848   03.10.2015          33      25     7460      299.00

```

```

      item_cnt_day  Month
0              1.0    01
1              1.0    01
2             -1.0    01
3              1.0    01
4              1.0    01
...      ...      ...
2935844          1.0    10
2935845          1.0    10
2935846          1.0    10
2935847          1.0    10
2935848          1.0    10

```

[2935849 rows x 7 columns]

```

[6]: def split_year(columns):
      word= columns.split('.')[2]
      return word

```

```
train_data['Year']=train_data['date'].apply(split_year)
```

```
[7]: train_data
```

```
[7]:
```

	date	date_block_num	shop_id	item_id	item_price	\
0	02.01.2013	0	59	22154	999.00	
1	03.01.2013	0	25	2552	899.00	
2	05.01.2013	0	25	2552	899.00	
3	06.01.2013	0	25	2554	1709.05	
4	15.01.2013	0	25	2555	1099.00	
...	...	...	...	...	...	
2935844	10.10.2015	33	25	7409	299.00	
2935845	09.10.2015	33	25	7460	299.00	
2935846	14.10.2015	33	25	7459	349.00	
2935847	22.10.2015	33	25	7440	299.00	
2935848	03.10.2015	33	25	7460	299.00	

	item_cnt_day	Month	Year
0	1.0	01	2013
1	1.0	01	2013
2	-1.0	01	2013
3	1.0	01	2013
4	1.0	01	2013
...	...	...	...
2935844	1.0	10	2015
2935845	1.0	10	2015
2935846	1.0	10	2015
2935847	1.0	10	2015
2935848	1.0	10	2015

[2935849 rows x 8 columns]

```
[8]: train_data['Sale']=train_data['item_price']*train_data['item_cnt_day']
```

```
[9]: sale_item
```

```
[9]:
```

	item_name	item_id	\
0	!	(.)	D 0
1	!ABBY FineReader 12 Professional Edition Full...		1
2	*** (UNV)	D	2
3	*** (Univ)	D	3
4	*** ()	D	4
...	...	...	...
22165	2 [PC,	] 22165	
22166	1 :	[] 22166	
22167	1 :	8 (+CD). ... 22167	
22168		Little Inu	22168

22169 () 22169

	item_category_id
0	40
1	76
2	40
3	40
4	40
...	...
22165	31
22166	54
22167	49
22168	62
22169	69

[22170 rows x 3 columns]

```
[10]: item_categories=[]
      for i in train_data['item_id']:
          item_categories.append(sale_item['item_category_id'].iloc[i])

      train_data['item_categories']=item_categories
```

```
[11]: train_data
```

```
[11]:
```

	date	date_block_num	shop_id	item_id	item_price	\
0	02.01.2013	0	59	22154	999.00	
1	03.01.2013	0	25	2552	899.00	
2	05.01.2013	0	25	2552	899.00	
3	06.01.2013	0	25	2554	1709.05	
4	15.01.2013	0	25	2555	1099.00	
...	...	...	...	...	...	
2935844	10.10.2015	33	25	7409	299.00	
2935845	09.10.2015	33	25	7460	299.00	
2935846	14.10.2015	33	25	7459	349.00	
2935847	22.10.2015	33	25	7440	299.00	
2935848	03.10.2015	33	25	7460	299.00	

	item_cnt_day	Month	Year	Sale	item_categories
0	1.0	01	2013	999.00	37
1	1.0	01	2013	899.00	58
2	-1.0	01	2013	-899.00	58
3	1.0	01	2013	1709.05	58
4	1.0	01	2013	1099.00	56
...	...	...	...	...	...
2935844	1.0	10	2015	299.00	55
2935845	1.0	10	2015	299.00	55

2935846	1.0	10	2015	349.00	55
2935847	1.0	10	2015	299.00	57
2935848	1.0	10	2015	299.00	55

[2935849 rows x 10 columns]

```
[12]: train_data['item_id_categories']=train_data['item_id'].apply(str) + ',' +
      ↪train_data['item_categories'].apply(str)
```

```
[13]: train_data
```

```
[13]:
```

	date	date_block_num	shop_id	item_id	item_price	\
0	02.01.2013	0	59	22154	999.00	
1	03.01.2013	0	25	2552	899.00	
2	05.01.2013	0	25	2552	899.00	
3	06.01.2013	0	25	2554	1709.05	
4	15.01.2013	0	25	2555	1099.00	
...	...	...	...	...	...	
2935844	10.10.2015	33	25	7409	299.00	
2935845	09.10.2015	33	25	7460	299.00	
2935846	14.10.2015	33	25	7459	349.00	
2935847	22.10.2015	33	25	7440	299.00	
2935848	03.10.2015	33	25	7460	299.00	

	item_cnt_day	Month	Year	Sale	item_categories	item_id_categories
0	1.0	01	2013	999.00	37	22154,37
1	1.0	01	2013	899.00	58	2552,58
2	-1.0	01	2013	-899.00	58	2552,58
3	1.0	01	2013	1709.05	58	2554,58
4	1.0	01	2013	1099.00	56	2555,56
...	...	...	...	...	...	...
2935844	1.0	10	2015	299.00	55	7409,55
2935845	1.0	10	2015	299.00	55	7460,55
2935846	1.0	10	2015	349.00	55	7459,55
2935847	1.0	10	2015	299.00	57	7440,57
2935848	1.0	10	2015	299.00	55	7460,55

[2935849 rows x 11 columns]

```
[14]: # 1. Grab your target variable FIRST while it still exists
training_target = train_data['item_cnt_day']

# 2. Drop the unwanted columns safely
columns_to_drop = ['date', 'date_block_num', 'item_price', 'Month', 'Year',
      ↪'Sale', 'item_id_categories', 'item_categories', 'item_cnt_day']
existing_drops = [col for col in columns_to_drop if col in train_data.columns]
X_train = train_data.drop(columns=existing_drops)
```

```
# 3. DIAGNOSTIC PRINT: Let's see if we actually have data left!
print("--- Data Check ---")
print(f"Number of rows in features (X_train): {X_train.shape[0]}")
print(f"Number of columns left: {X_train.shape[1]}")
print(f"Columns being used to predict: {list(X_train.columns)}")
```

```
--- Data Check ---
Number of rows in features (X_train): 2935849
Number of columns left: 2
Columns being used to predict: ['shop_id', 'item_id']
```

```
[15]: # 3. DIAGNOSTIC PRINT: Let's see if we actually have data left!
print("--- Data Check ---")
print(f"Number of rows in features (X_train): {X_train.shape[0]}")
print(f"Number of columns left: {X_train.shape[1]}")
print(f"Columns being used to predict: {list(X_train.columns)}")
```

```
--- Data Check ---
Number of rows in features (X_train): 2935849
Number of columns left: 2
Columns being used to predict: ['shop_id', 'item_id']
```

```
[17]: import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
import numpy as np

y_train = train_data['item_cnt_day']

columns_to_drop = [
    'date', 'date_block_num', 'item_price', 'Month', 'Year',
    'Sale', 'item_id_categories', 'item_categories', 'item_cnt_day'
]

X_train = train_data.drop(columns=columns_to_drop, errors='ignore')

print("--- Features Being Used to Train ---")
print(X_train.head())
print(f"Shape of X: {X_train.shape}, Shape of y: {y_train.shape}\n")

model = RandomForestRegressor(n_estimators=50, max_depth=6, random_state=42,
                               n_jobs=-1)
```

```

print("Training the Random Forest model... (this should be fast now)")
model.fit(X_train, y_train)
print("Training complete!")

# Evaluate
predictions = model.predict(X_train)
rmse = np.sqrt(mean_squared_error(y_train, predictions))
print(f"\nTraining RMSE Error: {rmse:.4f}")

```

--- Features Being Used to Train ---

	shop_id	item_id
0	59	22154
1	25	2552
2	25	2552
3	25	2554
4	25	2555

Shape of X: (2935849, 2), Shape of y: (2935849,)

Training the Random Forest model... (this should be fast now)
 Training complete!

Training RMSE Error: 2.4360

```

[19]: import pandas as pd
import numpy as np

# Group data by month (date_block_num), shop, and item to get monthly totals
monthly_train = train_data.groupby(['date_block_num', 'shop_id', 'item_id']).
    ↪agg({
        'item_cnt_day': 'sum'
    }).reset_index()

# Rename the column to reflect that it is now a monthly total
monthly_train.rename(columns={'item_cnt_day': 'item_cnt_month'}, inplace=True)

# Clip the values between 0 and 20 (Kaggle competition standard recommendation,
↪to handle outliers)
monthly_train['item_cnt_month'] = monthly_train['item_cnt_month'].clip(0, 20)

# . Bring back the item categories using our fast map trick
item_to_category = sale_item.set_index('item_id')['item_category_id']
monthly_train['item_category_id'] = monthly_train['item_id'].
    ↪map(item_to_category)

print("--- New Monthly Dataframe Created ---")
print(monthly_train.head())

```

```
print(f"\nNew shape of data: {monthly_train.shape}")
```

--- New Monthly Dataframe Created ---

	date_block_num	shop_id	item_id	item_cnt_month	item_category_id
0	0	0	32	6.0	40
1	0	0	33	3.0	37
2	0	0	35	1.0	40
3	0	0	43	1.0	40
4	0	0	51	2.0	57

New shape of data: (1609124, 5)

```
[ ]: lag features
```

```
[21]: def add_lag_features(df, lags):
        """Creates historical lag columns for item_cnt_month."""
        tmp = df[['date_block_num', 'shop_id', 'item_id', 'item_cnt_month']]

        for i in lags:
            shifted = tmp.copy()
            # Rename column to indicate how many months back it is looking
            shifted.columns = ['date_block_num', 'shop_id', 'item_id',
                                f'item_cnt_month_lag_{i}']
            # Shift the timeline forward so it matches up correctly when merged
            shifted['date_block_num'] += i
            df = pd.merge(df, shifted, on=['date_block_num', 'shop_id', 'item_id'],
                            how='left')
            # Fill missing values with 0 (if an item didn't sell X months ago, it
            # sold 0 units)
            df[f'item_cnt_month_lag_{i}'] = df[f'item_cnt_month_lag_{i}'].fillna(0)

        return df

# Let's generate lags for 1, 2, and 3 months ago
print("Generating lag features... (this will take a few seconds)")
monthly_train_lagged = add_lag_features(monthly_train, [1, 2, 3])

# Because lags look backward, the first 3 months (0, 1, 2) won't have complete
# history.
# We drop them so the model trains on perfectly clean historical data.
monthly_train_lagged = monthly_train_lagged[monthly_train_lagged['date_block_num'] >= 3]

print("Features built successfully!")
print(monthly_train_lagged.head())
```

Generating lag features... (this will take a few seconds)

Features built successfully!

	date_block_num	shop_id	item_id	item_cnt_month	item_category_id	\
187136	3	2	31	1.0	37	
187137	3	2	464	1.0	49	
187138	3	2	482	1.0	73	
187139	3	2	484	3.0	73	
187140	3	2	485	2.0	73	

	item_cnt_month_lag_1	item_cnt_month_lag_2	item_cnt_month_lag_3
187136	1.0	4.0	0.0
187137	0.0	0.0	0.0
187138	1.0	1.0	1.0
187139	2.0	4.0	2.0
187140	0.0	0.0	0.0

```
[23]: # Train set (Months 3 to 32)
X_train = monthly_train_lagged[monthly_train_lagged['date_block_num'] < 33].
        ↪drop(columns=['item_cnt_month'])
y_train = monthly_train_lagged[monthly_train_lagged['date_block_num'] <=
        ↪33]['item_cnt_month']

# Validation set (Month 33)
X_val = monthly_train_lagged[monthly_train_lagged['date_block_num'] == 33].
        ↪drop(columns=['item_cnt_month'])
y_val = monthly_train_lagged[monthly_train_lagged['date_block_num'] ==
        ↪33]['item_cnt_month']

print(f"Training features shape: {X_train.shape}")
print(f"Validation features shape: {X_val.shape}")
```

Training features shape: (1390457, 7)
Validation features shape: (31531, 7)

```
[25]: from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

# Initialize the model
model = RandomForestRegressor(n_estimators=30, max_depth=6, random_state=42,
        ↪n_jobs=-1)

print("Training model with lag features...")
model.fit(X_train, y_train)
print("Training complete!")

# Predict on Train and Validation sets
train_preds = model.predict(X_train)
val_preds = model.predict(X_val)

train_rmse = np.sqrt(mean_squared_error(y_train, train_preds))
```

```
val_rmse = np.sqrt(mean_squared_error(y_val, val_preds))

print("\n-----")
print(f"Training RMSE: {train_rmse:.4f}")
print(f"Validation RMSE (True Test): {val_rmse:.4f}")
print("-----")
```

Training model with lag features...
Training complete!

```
-----
Training RMSE: 2.0683
Validation RMSE (True Test): 2.1947
-----
```

[]: